

COE817 Lab 4

Objectives

- In this lab, you will work on a secure chat system. Netbeans is the IDE.
- Two students will work as a group.
- **Duration: two weeks.**

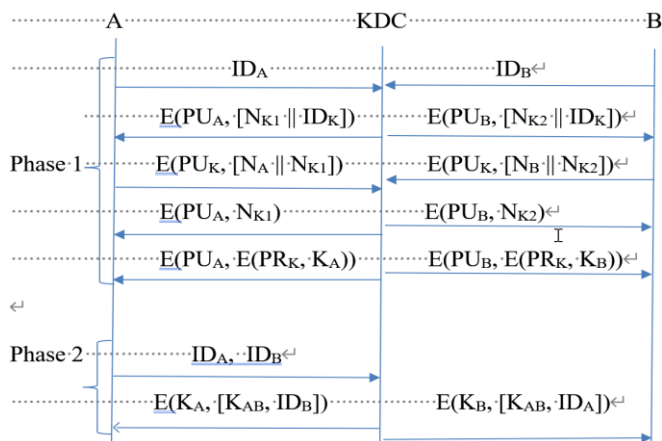
Write a Java socket communication program to implement a secure chat system. A chat server (KDC) is needed for distributing session keys to three clients (A, B and C). Any client could securely chat with the other two clients via KDC.

The required security features are:

- Authenticated key distribution between KDC and chat clients.
- All chat messages need to be encrypted using a block cipher such as DES or AES. A digital signature must be appended to all chat messages.
- Solution to combat replay attacks.

Key Distribution

This lab will use the same key distribution protocol as lab 3 (see the diagram below). Since there are 3 clients in the chat system, KDC needs to distribute a session key for each pair of clients. That is, K_{AB} (for A and B), K_{AC} (for A and C) and K_{BC} (for B and C). However, to ensure that any client's encrypted chat message could be correctly decrypted by the other two, we request that KDC generates and distributes a same session key K_s for AB, BC and AC ($K_s = K_{AB} = K_{BC} = K_{AC}$). The session key K_s is called a group key if it is shared by more than two parties.



Message Authentication and Encryption

To send a chat message, a client needs to implement the following two steps. For example, if A would like to send a message M to other clients (B and C),

- (1) The following protocol will be used to ensure chat message authentication and encryption

$A \rightarrow KDC: E(K_s, [ID_A, M]), Sig_A(ID_A, M)$

Here, message M and A's ID needs to be encrypted with the group key K_s and its signature also needs to be attached. The whole message will be sent to the KDC chat server. The server will then forward it to all other clients (B and C in this example). B and C will know that the message is from A based on the ID information of the message.

- (2) After receiving it, both B and C could be able to decrypt the message with K_s , verify digital signature and get the message M.

Similarly, if B or C would like to send a chat message, they need to follow the same two steps shown above.

This approach ensures all clients could chat via KDC. Any client who wants to send a message, send it to KDC and then KDC will correctly forward it to others.

Preventing Replay Attacks on Chat Messages

Even after you secure chat messages with encryption, there is still an obvious replay attack. For example, an attacker captures a chat message. He then repeatedly sends that message which is still a valid chat message. Flooding the intended recipient and making the chat room unusable for the participants. You need to improve the protocol in (1) and your solution should prevent an attacker from replaying a client's message.

What you needed to demonstrate to your TA

- Demonstrate your secure chat system with three messages sent by A, B and C.
 - First, allow your TA to type in a message M on client A. Your program needs to generate the message of protocol in (1) and display it. Then show TA if the same message M could be received by B and C after decryption and signature verification.
 - Secondly, allow your TA to type in a second message on client B and see if it could be received by A and C
 - Last, allow your TA to type in the third message on client C and see if it could be received by A and B
- Explain your code and tell how your KDC program could successfully forward a message to remaining clients. For example, if a chat message is from A, your program will ensure that KDC will forward A's message to B and C only.
- Demo your improved protocol and show how your solution could resist replay attack.

Submitting your lab

You must submit your source codes to lab4 submission folder on D2L at least 24 hours prior to the start of your scheduled lab period for project.